

Algorithms purpose: detect both overlapping communities and hierarchical structures.

Method: local optimization of fitness function

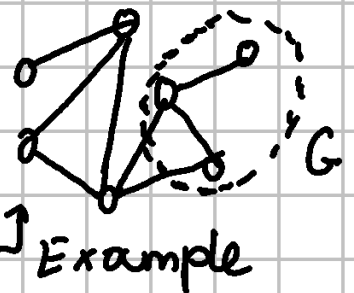
How community is detected: Peaks in fitness histogram.

* The parameters of the algorithm can be adjusted.

Basic formula (the Fitness function):

$$f_G = \frac{K_{in}^G}{(K_{in}^G + K_{out}^G)^\lambda}$$

* G - some module of the network



Where: K_{in}^G - total internal degree of nodes of G

K_{out}^G - total external degree of nodes of G

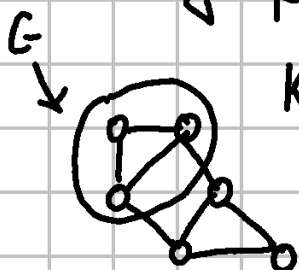
$\lambda \in \mathbb{R}^+$ → chosen parameter (can be changed)

In more detail:

$K_{in}^G = 2 \cdot \text{Number of links inside the module } G$

$K_{out}^G = \text{Number of links that connect module with the rest of the graph}$

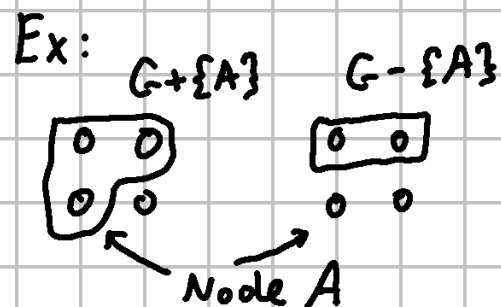
Example:



$$K_{in}^G = 2 \cdot 3 = 6$$

$$K_{out}^G = 2$$

New concept $\rightarrow$ fitness of node



$$f_G^A = f_{G + \{A\}} - f_{G - \{A\}}$$

Where : $G + \{A\}$ - module G with node A

$G - \{A\}$ - module of G without node A

The idea is that we pick some random G (not really random) and we start adding new nodes to it UNTIL every new node has negative value of f_G^A where A is one of the new nodes.

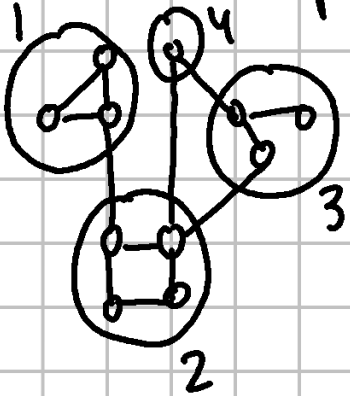
So, the algorithm for detecting community is as follows:

- 1) Pick node $I \rightarrow$ a community G
- 2) Iterate through each neighbor A_i and calculate $f_{G_i}^{A_i}$. Add neighbor with largest value $\rightarrow$ save new module as G'
- 3) Recalculate Fitness of each node in G'
- 4) If any node has negative fitness $\rightarrow$ it's removed from $G' \rightarrow$ save it as new subgraph G''

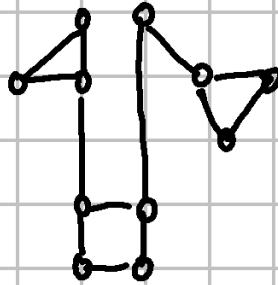
5) If 4 occurs, repeat from 3, o.w. repeat
1 for subgraph G''

We know that changing d changes the resolution
of communities, But we don't know 'a priori' how
large/small the communities are.

For example, the real communities are as follows:



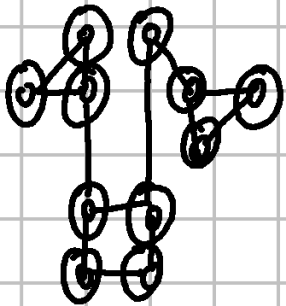
But, the only thing we see is
only



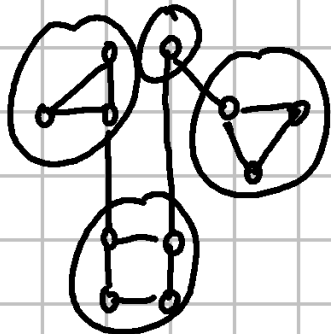
By changing
parameters d , we get different
sizes of 'estimated' communities

E.g.

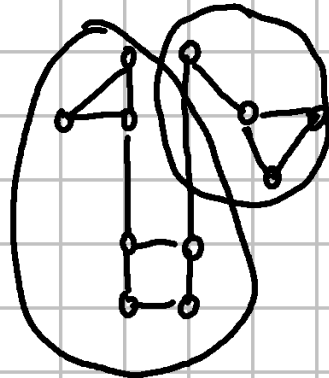
$d = 0.2$



$d = 1$



$d = 1.5$



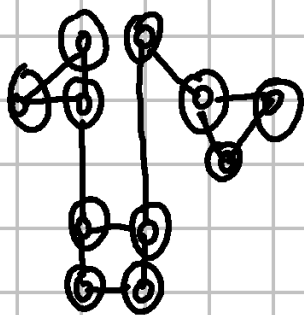
Let's call the distribution of communities for certain d a cover of the graph

Each d may produce different covers.

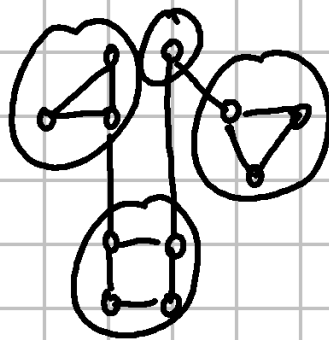
However, there might be certain ranges of d such that the cover stays the same.

Eg.

$0 < d \leq 0.2$



$0.4 < d \leq 1$



And so on ...

We see that cover #2 is "more stable" than cover #1

Now, let's label some cover of our network as 'P'.

Also, introduce function that calculates 'fitness' of communities of cover P

$$\bar{f}_P = \frac{1}{n_c} \sum_{i=1}^{n_c} f_{G_i} \quad (d=1) \quad \text{where } n_c \text{ is number of modules in cover P}$$

We calculate fitness for fixed value of d ,
(here we chose $d=1$) such that different values
of d produce same covers.